Documentation

Casualty and Disaster Insurance Management System

Submitted By:
Nanda Kishor Gudala

Capstone Project — The Hartford Training Program
GitHub: https://github.com/Nandakishorgudala/NANDKISHOR

Table of Contents

Table of Contents **2**
1. Abstract **4**
2. Problem Statement **5**
3. Project Objectives **6**
4. Technology Stack and Core Engines **7**
4.1. Server-Side Technologies (Backend) 7
4.2. Client-Side Technologies (Frontend) 8
5. System Architecture **9**
5.1. Backend Architecture Layers 9
5.2. Frontend Architecture 9
5.3. Interaction Flow Example (Raising a Claim) 10
6. Project Structure **11**
6.1. Backend Structure (Insurance/) 11
6.2. Frontend Structure (insurance-frontend) 11
7. Advanced Business Logic Engines **13**
7.1. Underwriting Engine (Risk Assessment) 13
7.2. Least Workload Assignment Algorithm 13
7.3. Coverage Category System 14
8. Detailed Module Explanations **15**
8.1. Admin Module 15
8.2. Customer Module 15
8.3. Agent Module 16
8.4. Claims Officer Module 16
9. Complete Business Workflow **17**
9.1. Policy Lifecycle: Registration to Activation 17
9.2. Claim Processing Lifecycle: Incident to Closure 17
10. Entity-Relationship (ER) Diagram and Database Design **19**
10.1. Core Entities 19
10.2. Entity Relationship Summary 20
11. Database Enumerations **21**
12. API Documentation Overview **22**
12.1. Authentication Endpoints (/api/Auth) 22
12.2. Customer Operations 22
12.3. Agent Operations 23
12.4. Claims Officer Operations (/api/Claims) 23
12.5. Administrative Operations 23
12.6. Support Services (/api/Documents, /api/Chat) 24
13. Frontend Component Architecture **25**

13.1. Core Services and State Management 25
13.2. Customer Dashboard Components 25
13.3. Agent Dashboard Components 26
13.4. Claims Officer Dashboard Components 26
13.5. Admin Console Components 26
13.6. Shared and Layout Components 27
14. Key Features and Technical Highlights 28
14.1. Key Features 28
14.2. Development Challenges and Solutions 28
15. Future Enhancements 30
16. Conclusion 31

1. Abstract

The Casualty and Disaster Insurance Management System (CDIMS) is a capstone project developed as part of The Hartford Training Program. It is an advanced, centralized digital platform engineered to modernize and streamline the traditional, multi-faceted processes associated with casualty and disaster insurance. Historically, the insurance sector has relied heavily on manual data entry, physical document handling, and localized data storage, leading to systemic inefficiencies, delayed claim processing, and frequent miscommunication among stakeholders.

The fundamental purpose of this system is to digitize and automate the entire insurance lifecycle — from initial customer onboarding, dynamic premium underwriting, and policy issuance to the complex orchestration of claim settlements involving multiple discrete roles. By implementing an end-to-end digital infrastructure, the system significantly reduces turnaround times, eliminates human error in premium calculations, and ensures transparent, traceable communication across all stakeholders.

The expected benefits encompass enhanced customer satisfaction through self-service portals, fortified data security through robust role-based authorization models, accelerated operational throughput, and comprehensive fraud prevention via mandatory document verification and multi-stage claim processing. Ultimately, this system provides a highly scalable, secure, and user-centric application empowering policyholders and insurance personnel with specialized, role-driven dashboards and automated workflows.

2. Problem Statement

The traditional casualty and disaster insurance industry suffers from several critical operational pain points that undermine efficiency, profitability, and customer trust:

- Excessive Manual Processing and Paperwork: A high reliance on physical documents for policy issuance and claim evidence collection leads to substantial administrative overhead, archiving costs, a high risk of document loss, and slow information retrieval times.
- Delayed Claim Settlements: The absence of integrated digital workflows means claims must be manually transitioned between customers, claims officers, and administrators, introducing severe bottlenecks and weeks-long resolution delays.
- Lack of Real-Time Verification for Disaster Severity: Traditional systems have no mechanism to assess regional disaster impact in specific geographic zones, resulting in inaccurate claim valuations and inconsistent payout decisions.
- Lack of Transparency and Traceability: Customers have no real-time visibility into the status of their policies or claims. Internally, tracking claim status or responsible officers becomes a managerial challenge.
- Complex Underwriting and Premium Calculation: Manual calculations for risk assessment based on asset data, geographic risk zones, and property age are inherently error-prone, leading to financial discrepancies and inconsistent pricing.
- Approval Bottlenecks and Security Risks: Without automated, role-based approval queues, policies and claims stagnate in backlogs. Physical files also lack granular access control, exposing sensitive data to unauthorized personnel.
- High Operational Overhead in Workload Assignment: Manual assignment of claims and policy requests to officers and agents leads to uneven workloads, reducing overall efficiency and increasing processing delays.
- Vulnerability to Fraudulent Claims: Without automated risk scoring, fraudulent claims are difficult to detect systematically, increasing financial losses for the insurer.

3. Project Objectives

The primary objectives of this system are to engineer a robust, enterprise-grade software ecosystem capable of automating and managing the full spectrum of casualty and disaster insurance activities:

1. Automation: Streamline policy applications from registration through activation, eliminating manual handoffs and reducing processing time.
2. Precision Underwriting: Implement an enhanced risk assessment engine to calculate premiums based on asset data, property age (YearBuilt), and geographic risk zones (High, Medium, Low) with configurable plan multipliers (Basic: 1x, Plus: 1.25x, Advanced: 1.5x).
3. Rapid Claims Processing: Enable Claims Officers to assess disaster claims quickly using interactive loss calculators, Regional Claims Density scoring by Zip Code, and Disaster Impact Scores.
4. Fraud Detection: Integrate an automated Fraud Risk Score engine that computes a fraud likelihood indicator for each claim based on incident data and regional patterns.
5. Transparency: Provide real-time dashboards for all stakeholders — Customers, Agents, Claims Officers, and Admins — each showing role-specific metrics and status updates.
6. Scalability: Build a modular, Clean Architecture-based system capable of handling multi-regional insurance operations without architectural changes.
7. Centralize Document Management: Establish secure digital upload and retrieval for KYC application proofs and disaster claim evidence, eliminating physical document dependency.
8. Implement Strict Role-Based Access Control (RBAC): Ensure data security and clear segregation of duties across four distinct organizational roles: Admin, Agent, Claims Officer, and Customer.

4. Technology Stack and Core Engines

The system is designed as a modern, full-stack web application utilizing a decoupled client-server architecture. It integrates several specialized engines and libraries to handle distinct functional requirements efficiently.

4.1. Server-Side Technologies (Backend)

Layer / Category	Technology	Purpose and Details
Framework	.NET 8.0 / ASP.NET Core Web API	Clean Architecture (Domain-Driven Design) with API, Application, Domain, and Infrastructure layers
ORM Engine	Entity Framework Core	Intuitive data access, schema migrations, and LINQ-based querying with SQL Server
Auth Engine	JSON Web Tokens (JWT)	Stateless, secure API communication with role-claim authorization (RBAC)
Database	Microsoft SQL Server	Structured, ACID-compliant relational data storage
Validation	FluentValidation	Robust server-side request validation
AI Integration	Microsoft Semantic Kernel	AI orchestration framework for future-assisted claim logic
Object Mapping	AutoMapper	Efficient convention-based mapping between domain entities and DTOs
API Docs	Swagger / OpenAPI	Interactive endpoint visualization, testing, and contract definitions
Testing	xUnit + Moq	Unit tests for controllers, services, and repositories

4.2. Client-Side Technologies (Frontend)

Layer / Category	Technology	Purpose and Details
Framework	Angular (Standalone Components)	Modular, reactive Single Page Application (SPA) with Standalone Components
State Management	Angular Signals	Modern, fine-grained reactive component state
Styling	Vanilla CSS	Premium aesthetic control and consistency
Visualization	Chart.js	Analytical dashboards and funnel visualizations

5. System Architecture

The project relies on a decoupled, multi-tiered RESTful architecture following Clean Architecture principles, logically divided into four distinct layers to promote separation of concerns and testability.

5.1. Backend Architecture Layers

- API Layer: Exposes RESTful endpoints and handles request/response formatting, JWT authentication, and global exception handling. Maps HTTP verbs to endpoint controllers and enforces [Authorize] attributes to verify JWT validity and RBAC permissions before routing to the service layer.
- Application Layer: Contains business logic interfaces, DTOs (Data Transfer Objects), service implementations, and the underwriting engine. This is where domain rules are enforced including premium calculation, risk scoring, Regional Claims Density computation, and workload balancing.
- Domain Layer: Defines core entities (User, PolicyApplication, Policy, InsuranceClaim, etc.), enumerations (UserRole, PolicyStatus, ClaimStatus), and fundamental domain logic that is independent of any infrastructure concerns.
- Infrastructure Layer: Implements data access using Entity Framework Core with SQL Server, file storage for uploaded documents, and the Microsoft Semantic Kernel AI integration.

5.2. Frontend Architecture

- Role-Based Dashboard Rendering: The Angular frontend dynamically configures navigation menus, accessible routes, and visible components based on the decoded JWT payload containing the user role.
- Reactive Data Management: Angular Signals manage component-level reactive state for instant UI updates, while RxJS Observables handle HTTP communication, stream transformations, and complex asynchronous event chains such as real-time claim status polling using interval and switchMap operators.
- Modular Component Design: Components are organized by role and feature, each communicating with backend services via Angular's HTTP client using strongly typed DTOs.

5.3. Interaction Flow Example (Raising a Claim)

```
Frontend (Angular)  →  POST /api/Claims + JWT
```

```
→  API Layer  →  Token & Role Validation
→  Claim Service (Verify Policy Active, Verify Coverage, Compute Regional
Density)
→  Repository (Insert Claim)  →  Database
→  Success Response  →  UI Update
```

6. Project Structure

6.1. Backend Structure (Insurance/)

Folder / Project	Contents
API/	Controllers (Admin, Agent, Auth, Customer, Claims, Documents, Chat) and global exception handling middleware
Application/	Business service interfaces and implementations, DTOs, repository interfaces, and Unit of Work pattern
Domain/	All database entities (Policy, Claims, User), enumerations, and fundamental domain logic
Infrastructure/	DbContext, EF Core migrations, repository implementations, Underwriting Engine, JWT configuration, and Semantic Kernel ChatBot service
Tests/	Unit and integration tests using xUnit and Moq for controllers, services, and repositories

6.2. Frontend Structure (insurance-frontend)

Folder	Contents
src/app/components/	Feature-specific components organized by role: admin/, agent/, customer/, officer-dashboard/
src/app/components/admin/	Admin dashboard, user management, analytics, and revenue report components
src/app/components/agent/	Agent dashboard, underwriting queue, and commission tracker components
src/app/components/customer/	Customer dashboard, policy marketplace, claim wizard, and nominee management
src/app/components/officer-dashboard/	Claims review queue, claim detail view, and settlement processor components
src/app/services/	API integration and business logic services: Auth (JWT), Policy, Claims, Store
src/app/models/	TypeScript interfaces and DTOs for strongly typed API communication

7. Advanced Business Logic Engines

7.1. Underwriting Engine (Risk Assessment)

The engine utilizes CalculateRiskScoreEnhanced logic to evaluate policy applications across multiple risk dimensions and produce a composite score determining eligibility and premium.

Risk Factor	Logic and Rule	Output
Asset Risk	Scoring based on value and age of property (YearBuilt)	Asset risk multiplier
Geographic Risk	Multipliers based on Risk Zones (High / Medium / Low)	Geographic multiplier
Plan Multiplier	Basic: 1x, Plus: 1.25x, Advanced: 1.5x	Plan adjustment
Final Risk Score	Product of all multipliers applied to composite score	FinalRiskScore
Premium Calculation	Premium = BasePremium × FinalRiskScore × PlanMultiplier	CalculatedPremium

7.2. Least Workload Assignment Algorithm

Used during Agent and Claims Officer assignment. The system queries and selects active staff members sorted by their current assigned task count, ensuring balanced load distribution across the team.

Step	Description
1	Fetch the count of currently active (pending or in-review) tasks for each eligible agent or claims officer
2	Sort the staff list in ascending order by active task count
3	Assign the incoming request to the staff member at the top of the sorted list (minimum workload)
4	Persist the assignment in the database and send an in-app notification to the assigned staff member

Formula: activeAgents.OrderBy(a => a.AssignedCount).First()

7.3. Coverage Category System

Policy coverages are dynamic based on the PolicyProduct (e.g., Casualty, Disaster). Deductibles are flexible and set at the time of application. Remaining Limit is automatically tracked in real-time as multiple claims are processed.

Coverage Category	Description	Payout Basis
Casualty Protection	Coverage for bodily injury resulting from accidents	Lump sum based on injury severity
Disaster Recovery	Coverage for property loss due to natural or man-made disasters	Property Loss % × Coverage Amount
Structural Damage	Coverage for structural damage to insured assets	Assessed damage % of coverage
Business Interruption	Compensation for loss of income during recovery period	Weekly compensation for recovery period

8. Detailed Module Explanations

The system employs granular Role-Based Access Control to partition functionality across four core user groups, ensuring that users interact only with data and operations pertinent to their organizational role.

8.1. Admin Module

The Admin is the super-user responsible for system configuration, user management, and macro-level oversight of the entire platform.

- Agent and Officer Registration: Creates and provisions new Agent and Claims Officer accounts with appropriate role assignments and access credentials.
- Workload-Based Assignment: Uses the Least Workload Algorithm to automatically assign incoming policy requests to agents and new claims to claims officers.
- Analytics Dashboard: Access to aggregated data visualizations detailing system health, total revenue collected, claim resolution ratios, and policy distribution metrics.
- Revenue Reports: Detailed financial breakdowns of premiums collected, claims settled, agent commissions earned, and overall business performance.
- Policy and User Management: Global oversight of all registered users, active policies, pending requests, and system-wide claims.

8.2. Customer Module

The Customer is the primary end-user seeking casualty and disaster insurance coverage and the initiator of the claims process.

- Policy Marketplace: Browse available insurance products with detailed coverage descriptions, premium estimates, and plan comparisons.
- Policy Application: Multi-step form submission including asset information, geographic location (Zip Code), property age, and supporting document upload for underwriting assessment.
- Premium Payment: Record premium payments to activate policies following successful underwriting approval.
- Claim Filing: Guided wizard for reporting incidents, selecting the coverage category, specifying the claim amount, and uploading supporting evidence documents.
- Claim Tracking and Payout Acceptance: View claim status, officer notes, and accept final settlement payouts.
- Notifications and History: View policy status updates, claim progress, payment history, and renewal reminders.

8.3. Agent Module

The Agent is the intermediary representative who assists customers with policy acquisition and performs underwriting assessments on assigned policy requests.

- Assigned Request Queue: A prioritized queue displaying policy requests assigned to this agent for risk evaluation.
- Risk Assessment Interface: Input evaluation factors for each application and trigger the Underwriting Engine to compute the risk score, determine eligibility, and calculate the recommended premium.
- Direct Policy Initiation: Ability to initiate a policy application on behalf of a customer, streamlining the onboarding process.
- Commission Summary: Track total commissions earned from successfully activated policies.

8.4. Claims Officer Module

The Claims Officer is the final decision-maker responsible for reviewing, assessing, approving or rejecting insurance claims, and processing settlements.

- Review Queue: Streamlined interface for managing the queue of assigned insurance claims with status indicators and claim amounts.
- Disaster Risk Assessment Panel: View Regional Claims Density for the incident Zip Code, input Disaster Impact Score and Property Loss %, and review computed Fraud Risk Score and Suggested Payout.
- Interactive Loss Calculator: What-if analysis tool allowing officers to evaluate different payout scenarios before finalizing the decision.
- Settlement Processing: Define the final approved payout amount and initiate settlement processing for approved claims.

9. Complete Business Workflow

9.1. Policy Lifecycle: Registration to Activation

9. Customer Onboarding: A new user self-registers creating a User profile with Customer privileges, providing essential biographical details and account credentials.
10. Policy Marketplace Browsing: The customer browses available policy types and their coverage categories, reviewing base premiums and coverage amounts.
11. Policy Application Submission: The customer, or an Agent on their behalf, submits a policy request including asset details, Zip Code, property age (YearBuilt), geographic risk zone, and supporting documents.
12. Underwriting Assessment: An assigned Agent performs the underwriting assessment. The system evaluates asset risk, geographic zone multipliers, and plan type to compute a composite risk score and calculate the personalized premium.
13. Eligibility Decision: Applications that exceed the risk threshold are automatically flagged and the customer is notified with the reason. Eligible requests proceed to the payment stage.
14. Premium Payment and Activation: Upon successful premium payment, the system generates an active Policy record with a unique Policy Number, defined coverage amounts, start and end dates, and sends a confirmation to the customer.
15. Policy Lifecycle Management: Active policies generate renewal reminders as the expiry date approaches. Upon expiry, customers can submit renewal requests to continue coverage.

9.2. Claim Processing Lifecycle: Incident to Closure

16. Claim Initiation: The customer raises a claim against an Active policy, providing incident details, selecting the coverage category, specifying the claim amount, uploading supporting documents, and submitting the incident Zip Code. Claim status is set to Submitted.
17. Regional Claims Density Indexing: The system calculates the Regional Claims Density for the provided Zip Code to contextualise the severity of the incident geographically.
18. Administrative Assignment: Admin assigns the claim to a Claims Officer using the Least Workload Algorithm, ensuring equitable distribution across the officer team.
19. Document Verification: The Claims Officer reviews uploaded evidence and incident details. If documents are insufficient, additional documentation is requested from the customer.

20. Claim Assessment: The Officer reviews the Regional Claims Density, inputs the Disaster Impact Score, Property Loss %, and any fraud risk indicators. The system computes the Fraud Risk Score and Suggested Payout.
21. Decision and Approval: For approved claims, the Officer enters the final approved amount and initiates settlement processing. The claim status transitions to Approved.
22. Customer Acceptance: Customer reviews the settlement offer and accepts. The system issues the final Payout Advice Invoice and automatically deducts the approved amount from the policy's remaining coverage limit.
23. Settlement: Claim status transitions to Settled. For rejected claims, the officer records a rejection reason and the customer is notified accordingly.

10. Entity-Relationship (ER) Diagram and Database Design

The database design is based on a fully normalized relational schema designed to Third Normal Form (3NF) to ensure data integrity, eliminate redundancy, and accurately track all lifecycle state transitions. All primary keys are GUIDs. Status fields use strongly typed enumerations stored as integers.

10.1. Core Entities

Entity	Description
User	Central identity record for all roles. Unified table differentiated by Role enum (Admin, Agent, Customer, ClaimsOfficer). Stores bcrypt-hashed passwords, contact details, and underwriting-relevant fields (Occupation, DOB). IsActive flag supports soft-delete without data loss.
PolicyProduct	Master catalog of insurance products. Defines Name, BasePremium, coverage limits, tenure, and plan multipliers (Basic, Plus, Advanced). Created and managed by Admins.
PolicyApplication	Represents a customer's application for coverage. Captures asset data, geographic zone, Zip Code, document path, and computed TotalRiskScore and CalculatedPremium after underwriting assessment.
Policy	Active insurance contract generated after application approval and payment. Contains unique PolicyNumber, StartDate, EndDate, FinalPremium, CoverageAmount, and RemainingLimit tracked in real time across multiple claims.
InsuranceClaim	Claim filed against an active policy. Records IncidentDate, Description, ClaimAmount, ApprovedAmount, FraudRiskScore, DisasterImpactScore, RegionalClaimsDensity, and full status lifecycle from Submitted to Settled.
Invoice	Financial record for both incoming premiums (InvoiceType: Premium) and outgoing payouts (InvoiceType: Claim). Contains Amount, DueDate, and PaymentStatus.
Payment	Tracks individual financial transactions per policy. Supports retries and partial payment patterns. A Paid status triggers Policy activation from Draft to Active.
Commission	Agent earnings calculated upon successful policy activation. Linked to Policy and Agent User records.
Nominee	Extension point for policy beneficiaries. Policyholder can designate multiple nominees per active policy for different coverage categories.

10.2. Entity Relationship Summary

Entity	Relationship	Cardinality

User (Customer)	Submits	1 Customer → Many PolicyRequests
User (Agent)	Evaluates	1 Agent → Many PolicyRequests
User (ClaimsOfficer)	Reviews	1 Officer → Many InsuranceClaims
PolicyProduct	Defines terms for	1 PolicyType → Many PolicyRequests
PolicyRequest	Results in	1 PolicyRequest → 1 Policy (1:1)
Policy	Subject of	1 Policy → Many InsuranceClaims
InsuranceClaim	Produces	1 InsuranceClaim → 1 Settlement (on approval)
Invoice	Records	Financial records for Premiums (In) and Payouts (Out)

11. Database Enumerations

All status fields in the system use strongly typed enumerations stored as integers in the database. This ensures referential integrity, prevents invalid state values, and enables the state-machine logic enforced by the service layer.

Enumeration	Values	Used In
ApplicationStatus	Pending, Assigned, Approved, Rejected	PolicyApplication lifecycle
PolicyStatus	Active, Expired, Lapsed, Cancelled	Policy lifecycle tracking
ClaimStatus	Submitted, UnderReview, Approved, Rejected, Settled	InsuranceClaim pipeline
InvoiceType	Premium, Claim	Invoice financial records
RiskCategory	Low, Medium, High	Underwriting risk zones
UserRole	Admin, Agent, Customer, ClaimsOfficer	RBAC and dashboard routing

12. API Documentation Overview

The ASP.NET Core API provides highly structured, REST-compliant endpoints secured by JWT authentication with Role-Based Access Control (RBAC) enforcement. All endpoints require a valid JWT token in the Authorization header except the Authentication endpoints. Interactive documentation and endpoint testing are available via Swagger / OpenAPI.

12.1. Authentication Endpoints (/api/Auth)

Method	Endpoint	Description
POST	**/api/Auth/register**	Register a new customer account with secure password hashing.
POST	**/api/Auth/login**	Authenticate user credentials and receive a signed JWT token with role claims.
POST	**/api/Auth/forgot-password**	Initiate a password reset process for registered users.

12.2. Customer Operations (/api/Customer, /api/PolicyApplications, /api/Claims, /api/payments)

Method	Endpoint	Description
POST	**/api/Customer/profile**	Create a customer profile with identity details (required before policies).
GET	**/api/Customer/profile**	Retrieve the current user's profile information.
POST	**/api/PolicyApplications**	Submit a new insurance policy application (Standard).
POST	**/api/PolicyApplications/apply-with-plan**	Submit an application with a specific coverage plan (Basic, Plus, Advanced).
GET	**/api/PolicyApplications/customer**	View status of own policy applications and underwriting results (Risk Score).
POST	**/api/Claims**	File a new insurance claim with incident details and attached proof.
GET	**/api/Claims/my-claims**	List all historical and current claims submitted by the customer.
POST	**/api/Claims/{id}/accept**	Finalize and accept an approved claim settlement amount.

POST	**/api/payments/pay-policy**	Pay premium invoice to activate an approved policy application.

12.3. Agent Operations (/api/PolicyApplications, /api/Dashboard)

Method	Endpoint	Description
GET	**/api/PolicyApplications/agent**	List policy requests assigned to this agent for review and endorsement.
POST	**/api/PolicyApplications/{id}/approve**	Endorse an application for approval after manual review of asset data.
POST	**/api/PolicyApplications/{id}/reject**	Reject an application with a specified reason code.
GET	**/api/PolicyApplications/agent/customers**	View customers and active policies currently managed by the agent.
GET	**/api/Dashboard/agent/stats**	View performance metrics, including pending reviews and approved counts.

12.4. Claims Officer Operations (/api/Claims)

Method	Endpoint	Description
GET	**/api/Claims/officer/assigned**	List all individual claims assigned to this officer for professional review.
POST	**/api/Claims/review**	Submit a detailed assessment (Disaster Impact vs. Property Loss).
POST	**/api/Claims/approve**	Submit a decision to approve a specific payout amount.
POST	**/api/Claims/reject**	Submit a decision to reject a claim with detailed justification.

12.5. Administrative Operations (/api/admin, /api/Dashboard)

Method	Endpoint	Description
POST	**/api/admin/create-agent**	Provision a new Agent account with role assignments and branch data.
POST	**/api/admin/staff**	Generic endpoint to provision Agents or Claims Officers.
POST	**/api/admin/assign-agent**	Manually assign or override agent assignment for an application.

POST	**/api/admin/assign-officer**	Manually assign or reassign a claim to a specific Claims Officer.
GET	**/api/Dashboard/admin/stats**	Retrieve platform-wide metrics (Revenue, Pending Apps, Active Policies).
GET	**/api/admin/agent-performance**	Analytics on agent efficiency, commissions, and conversion rates.
GET	**/api/admin/officer-performance**	Breakdown of officer workload and claim resolution ratios.
GET	**/api/Dashboard/admin/charts**	Detailed time-series data for policies, claims, and revenue trends.

12.6. Support Services (/api/Documents, /api/Chat)

Method	Endpoint	Description
POST	**/api/Documents/upload**	Upload KYC/Identity documents supporting a policy application.
POST	**/api/Documents/claim/upload**	Upload high-resolution images or PDF proof for a disaster claim.
GET	**/api/Documents/{id}**	Securely stream an application document (RBAC restricted).
GET	**/api/Documents/claim/{id}**	Securely stream a claim-related proof document.
POST	**/api/Chat**	Interact with the Semantic Kernel-powered AI assistant for general queries.

13. Frontend Component Architecture

The frontend is built using Angular with Standalone Components following a modular, role-based architecture. State is managed using Angular Signals for component-level reactivity and RxJS Observables for all asynchronous data streams and service communication.

13.1. Core Services and State Management

Service	Technology	Responsibility
Auth	Angular Signals + RxJS	Manages JWT storage, decodes role claims, handles session lifecycle, and exposes a reactive currentUser signal consumed across all dashboards
PolicyRequest	RxJS Observables	Handles all HTTP communication for policy applications and underwriting updates using switchMap and catchError operators
Claim	RxJS BehaviorSubject	Manages the claim lifecycle from creation to settlement, using BehaviorSubject to track current claim state across components
Notification	RxJS interval + HttpClient	Polls the notifications endpoint at regular intervals using RxJS interval and mergeMap, delivering real-time alerts to the NotificationBell component
File	RxJS + HttpClient	Handles multipart document uploads using RxJS for progress tracking and error recovery

13.2. Customer Dashboard Components

Component	Description
PolicyMarketplace	Interactive card-based display of available insurance products with coverage details and premium estimates
PolicyApplication	Multi-step form wizard for submitting policy requests including asset data, Zip Code, and document upload
MyPolicies	Centralized view of active policies, coverage details, nominee information, and payment history
ClaimWizard	Guided step-by-step form for reporting incidents, selecting coverage category, and uploading evidence
NomineeManagement	Interface to add, edit, or remove nominees for each active policy

13.3. Agent Dashboard Components

Component	Description
UnderwritingQueue	Prioritized list of policy requests assigned to this agent for risk evaluation
RiskAssessment	Interface for entering risk evaluation parameters and viewing the system-calculated risk score and premium
CommissionTracker	Dashboard summarizing commissions earned from activated policies
FunnelComponent	D3.js / Chart.js integration for sales funnel visualization: Leads → Proposals → Policies

13.4. Claims Officer Dashboard Components

Component	Description
ClaimReviewQueue	Tabular list of assigned claims with status indicators, Fraud Risk Scores, and claim amounts
ClaimDetail	Full detail view including incident description, uploaded documents, policy coverage summary, and review history
DisasterRiskPanel	Visualization of Regional Claims Density vs. Disaster Impact Score for fraud vs. impact analysis
SettlementProcessor	Form for entering the final approved settlement amount and submitting the settlement decision

13.5. Admin Console Components

Component	Description
UserManagement	CRUD interface for Agent and Claims Officer accounts with workload display
SystemOverview	Global summary of all active policies, pending requests, and open claims
AnalyticsDashboard	Visual dashboards for revenue, claim metrics, and business performance using Chart.js
RevenueReport	Detailed financial reports with date range filters for premiums collected and claims settled

13.6. Shared and Layout Components

Component	Description

AppHeader	Dynamic navigation bar adapting its menu items based on the decoded JWT role claim
AppSideNav	Role-specific sidebar for dashboard sub-navigation with active route highlighting
DocumentViewer	Secure iFrame integration for rendering uploaded PDF and image proof documents inline
ChatBot	Floating AI assistant powered by Microsoft Semantic Kernel, available across all dashboards
PublicPage	Landing page consisting of testimonials, tour component, and product advertisement

14. Key Features and Technical Highlights

14.1. Key Features

- Stateless JWT Authentication and RBAC: A highly secure, stateless API architecture where the frontend dynamically configures navigation based on JWT role claims and every backend endpoint enforces [Authorize(Roles=...)] protection.
- Region-Aware Scoring: Automatic Regional Claims Density tracking via incident Zip Code feeds directly into the Disaster Impact assessment, enabling geographically contextualised claim evaluations.
- Calculated Risk: Underwriting is not static — it adjusts premiums based on multiple asset factors, geographic risk zones, and configurable plan multipliers.
- Fraud Risk Engine: Automated Fraud Risk Score computed for each claim based on disaster impact vs. property loss patterns, flagging suspicious submissions for officer attention.
- Least Workload Assignment Algorithm: Automated, equitable distribution of policy requests and claims based on real-time active workload metrics, eliminating manual assignment overhead.
- Modern Angular Architecture: Zero-dependency on legacy lifecycle hooks, using 100% Standalone Components and Signals architecture for fine-grained reactivity.
- Multi-Stage Claim Processing Pipeline: Declarative state-machine logic enforces valid claim status transitions — document verification, officer investigation, and settlement processing are mandatory sequential steps that cannot be bypassed.
- Configurable Coverage Category System: Each policy type defines multiple coverage categories with independently configured payout bases and deductible options.

14.2. Development Challenges and Solutions

Challenge	Solution Implemented
Multi-Role Access Control	Implemented strict RBAC with JWT role claims and [Authorize(Roles=...)] on every endpoint. Angular RoleGuard prevents unauthorized route access. Angular Signals expose a reactive currentUser to all dashboard components.
Dynamic Underwriting Engine	Abstracted risk evaluation into independent evaluator classes following the Strategy pattern, making the engine easily extensible with new risk dimensions without modifying existing code.
Workload Balancing Without Race Conditions	Assignment logic is wrapped in a database transaction with optimistic concurrency control to prevent duplicate assignments when multiple requests arrive simultaneously.

Document Upload and Storage	Multipart file uploads with server-side validation store files in wwwroot folder in backend and persist only file paths in the database to avoid BLOB performance degradation.
Complex Asynchronous Frontend Flows	RxJS operators (switchMap, combineLatest, debounceTime) were used to compose complex asynchronous workflows including dependent API calls, real-time polling, and concurrent multi-file uploads predictably.

15. Future Enhancements

The architecture of this system is designed for extensibility, allowing seamless integration of advanced enterprise features in subsequent phases:

- AI Automation: Automated claim rejection for claims with Fraud Risk Score exceeding 90%, integrated with the Microsoft Semantic Kernel pipeline to reduce officer workload on obvious fraud cases.
- Mobile Integration: Progressive Web App (PWA) support enabling on-site incident reporting with photo capture, GPS-tagged incident location, and offline-first form submission for field agents and customers.
- Blockchain Payouts: Instant settlement using smart contracts on a private blockchain for verified disaster events, enabling trustless, automatic payout disbursement without manual officer approval for pre-verified disaster zones.
- Real Payment Gateway Integration: Replace simulated premium payment flows with live integration to Razorpay, Stripe, or PayPal webhooks for real-time payment processing and automated reconciliation.
- Email and SMS Notification Service: Integrate production-grade notification providers (SendGrid for email, Twilio for SMS) using background worker queues for reliable asynchronous lifecycle event delivery.
- Advanced Analytics and Business Intelligence: Integration of a BI module with predictive analytics for regional claim forecasting, agent performance scoring, and portfolio risk modelling powered by historical claims data.
- Document OCR and Verification: Implement optical character recognition to automatically extract and validate information from uploaded identity documents, property titles, and disaster certificates.
- Policy Renewal Automation: Build automated renewal workflows with configurable reminder schedules and pre-filled renewal forms based on existing policy data.

16. Conclusion

The Casualty and Disaster Insurance Management System (CDIMS) represents a modern approach to specialized insurance, prioritizing data accuracy and rapid disaster response. By combining a robust .NET 8.0 Clean Architecture backend with specialized business engines — including the Region-Aware Underwriting Engine, the Fraud Risk Score system, and the Least Workload Assignment Algorithm — it provides a stable and scalable foundation for insurance providers operating in disaster-prone and casualty-intensive regions.

The platform successfully demonstrates the principles of modern software engineering applied to complex enterprise business logic in the insurance domain. By systematically replacing manual operations with secure, scalable, and rigidly role-driven digital workflows, this system represents a highly viable solution for optimizing casualty and disaster insurance operations at scale.

The full source code is available at: https://github.com/Nandakishorgudala/NANDKISHOR

Prepared by: Nanda Kishor Gudala
Capstone Project — The Hartford Training Program | March 2026